Convolutional Networks, and Course Synthesis

Lesson 10 — Technologies for Artificial Intelligence

Fabio Antonini — Università degli Studi dell'Aquila

18 December 2026 · reading time about 100 minutes

Contents

Lesson plan . . . 1
1. Why this lesson exists . . . 2
    1.1 Meridian's wafers . . . 3
2. One neuron per pixel is the wrong shape . . . 3
3. The convolution . . . 4
    3.1 The operation . . . 4
    3.2 How big is the output? . . . 4
    3.3 Kernels chosen by hand, and the zero-sum trick . . . 5
4. Equivariance, and what pooling turns it into . . . 6
    4.1 The property the whole idea rests on . . . 6
    4.2 Pooling: from *where* to *whether* . . . 7
5. A convolutional network, and what it costs . . . 8
    5.1 Head to head . . . 8
6. The experiment this lesson exists for . . . 9
    6.1 The predictable mistake . . . 10
7. Augmentation against architecture . . . 11
8. What the assumption is worth, measured in data . . . 12
9. What is the convolution actually using? . . . 13
10. Transfer learning . . . 15
    10.1 The source task is the whole game . . . 16
    10.2 A window, not a slope . . . 16
    10.3 When transfer makes things worse . . . 17
11. A note on what this lesson did not need . . . 17
12. Ten lessons on one problem . . . 18
    12.1 Choosing, in one table . . . 19
    12.2 And after this . . . 19
Summary . . . 20
    Homework . . . 20
    Notation used in this lesson . . . 20
Further reading . . . 21

Lesson plan

Time	Minutes	Segment	Material
0:00–0:10	10	Exercise 9 discussed; the promise lesson 9 left open	Slides 2–7
0:10–0:28	18	Meridian's wafers; why a dense layer is the wrong shape	Slides 8–16
0:28–0:45	17	The convolution: definition, output size, kernels by hand	Slides 17–26
0:45–1:05	20	**Notebook 01** — convolution from scratch	Slide 27
1:05–1:17	12	**Break**	Slide 28
1:17–1:35	18	Equivariance, pooling, and the architecture	Slides 29–37
1:35–1:50	15	A defect where none has been seen; augmentation	Slides 38–45
1:50–2:12	22	**Notebook 02** — convolutional networks on wafers	Slide 46
2:12–2:25	13	Transfer learning, and when it makes things worse	Slides 47–54
2:25–2:43	18	**Notebook 03** — transfer and synthesis	Slide 55
2:43–2:57	14	Ten lessons on one problem; where the field goes	Slides 56–63
2:57–3:00	3	Homework, and the close	Slides 64–65
	180	**Total**	**65 slides, 3 notebooks**

---

1. Why this lesson exists

Lesson 9 ended with a claim and no demonstration:

> Everything here treats the 64 pixels of a digit as 64 unrelated numbers — permute them consistently across the dataset and nothing in this lesson notices.

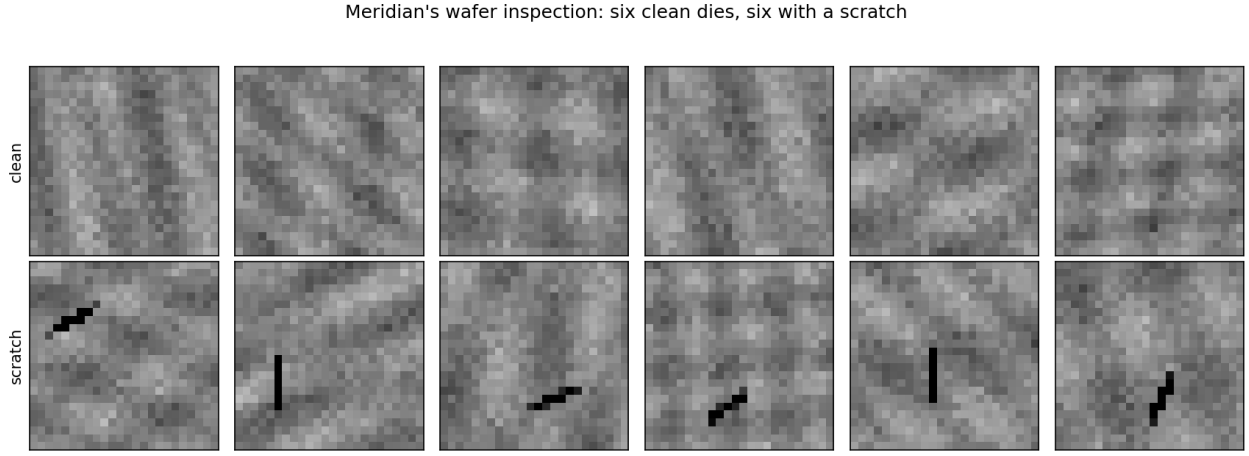
This lesson is about what you get by refusing to throw that arrangement away, and it is the last of the course, so it also has to add the ten weeks up.

The answer is not "convolutional networks are better at images". Section 12 runs every family the course has met on one problem and finds a random forest, given raw pixels and no notion

of adjacency whatsoever, within two thousandths of the convolutional network. The answer is something more useful and less comfortable, and it is about **representations**.

1.1 Meridian's wafers

Meridian Instruments, whose optical sensors ran through lesson 9, cuts them from silicon wafers. Each die is photographed at 24×24 pixels and graded, and a die is rejected when it carries a **defect**: a scratch about seven pixels long, or a bright contaminating particle.



Six clean dies and six carrying a scratch. The defect occupies a handful of the 576 pixels, and the surrounding film thickness varies smoothly, so absolute brightness says nothing — a scratch is dark relative to its immediate surroundings, not dark in absolute terms.

Two properties of this problem decide everything that follows, and both are visible in that figure.

The defect is local. It concerns a few adjacent pixels and the rest of the die is irrelevant.

The defect means the same thing wherever it lands. A scratch in the top left and a scratch in the bottom right are the same event, graded the same way.

The grading station is imperfect at a published rate of 2%, which puts a ceiling on this lesson exactly as lesson 9's test rig did. The batch used in notebook 01 drew 2.55%. Over 4,000 dies the number of wrong grades is binomial, with a standard deviation for the rate of $\sqrt{0.02 \times 0.98/4000} = 0.0022$, so $(0.0255 - 0.02)/0.0022 = 2.5$ standard deviations high — a high draw, not a bug, and the reason every score below is read against the ceiling **realised on the set it was measured on** rather than against the design figure.

> **Try this:** before reading further, look at the twelve dies above and decide what you would compute from an image to grade it. If your answer involves a small window and the word "anywhere", you have derived the content of sections 3 and 5.

---

2. One neuron per pixel is the wrong shape

A dense layer connects every input to every unit. On a 24×24 image that is 576 weights per unit, and here is the consequence that matters: **a unit that has learned to spot a scratch at one**

position knows nothing about the same scratch three pixels to the left. Those are different weights, and they were trained by different examples.

layer	parameters
dense, 256 units	147,712
dense, 1024 units	590,848
convolutional, 8 kernels of 3×3	**80**
convolutional, 16 kernels of 3×3	160
convolutional, 32 kernels of 5×5	832

Eighty against a hundred and forty-seven thousand — 147,632 fewer, a ratio of 1,846. But compression is the least interesting thing here, and reading the table that way misses the point twice over.

First, the convolutional layer **produces more numbers than the dense one**, not fewer: eight feature maps of 24×24 is 4,608 outputs against 256. It is not a smaller layer.

Second, and this is the whole idea: the saving comes from using *the same* nine weights at every position. That is **weight sharing**, and what it buys is not memory but **evidence**. One example of a scratch anywhere on any die teaches the detector about scratches everywhere. The dense layer has to be taught position by position, and section 9 measures what that costs in data.

---

3. The convolution

3.1 The operation

A convolutional layer slides a small array of weights — the **kernel** — over the image and, at each position, writes down the weighted sum of the pixels it covers:

$$(I * K)[r, c] = \sum_{i=0}^{f-1} \sum_{j=0}^{f-1} I[r+i,\ c+j]\ K[i, j]$$

Strictly this is *cross-correlation*; a true convolution flips the kernel first. Every deep learning library computes the expression above and calls it convolution, and because the kernel is learned rather than given, the flip changes nothing that matters. This course follows the library.

Notebook 01 implements it in eleven lines and checks it against `scipy`, which is an independent implementation by other people: the largest disagreement is 8.9×10^{-16}, which is floating-point rounding and not an error.

3.2 How big is the output?

With an $n \times n$ input, an $f \times f$ kernel, padding p and stride s:

$$\text{output size} = \left\lfloor \frac{n + 2p - f}{s} \right\rfloor + 1$$

It is worth counting rather than memorising, because every part of the expression is one step of the count. Padding adds p pixels at each end, so a row is $n + 2p$ pixels long. A window placed at position r covers pixels r through $r + f - 1$, so it fits only while $r + f - 1 \leq n + 2p - 1$: the **last legal start is** $n + 2p - f$. Starting at 0 and stepping by s, the starts are $0, s, 2s, ...$, and the number of them that do not pass that last one is $\lfloor (n + 2p - f)/s \rfloor$ — plus one, because the count includes the start at zero.

So the +1 is the window at position zero, and the floor is doing real work: when the stride does not divide the available positions evenly, the last incomplete window is simply not taken.

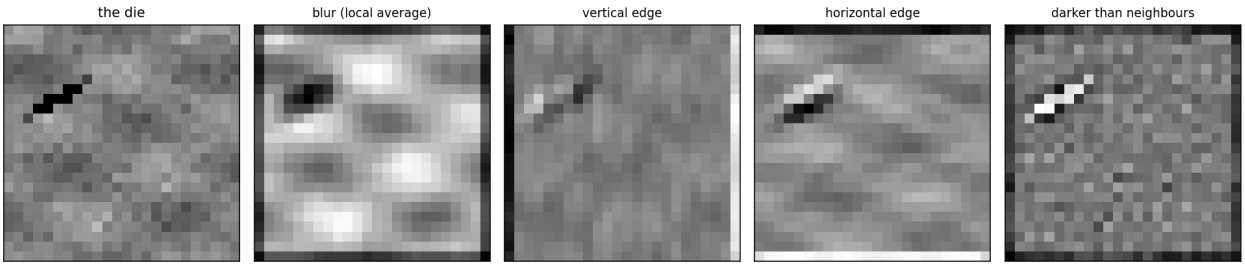
Worked, on this lesson's images. With $n = 24$:

f	p	s	formula	measured
3	0	1	$(24 + 0 - 3)/1 + 1 = 22$	22
3	1	1	$(24 + 2 - 3)/1 + 1 = 24$	24
5	0	1	$(24 + 0 - 5)/1 + 1 = 20$	20
5	2	1	$(24 + 4 - 5)/1 + 1 = 24$	24
3	1	2	$\lfloor (24 + 2 - 3)/2 \rfloor + 1 = 12$	12

Two cases cover almost everything you will write. **A 3×3 kernel with $p = 1$ leaves the size unchanged** — this is what Keras calls `padding="same"`, and the general rule is $p = (f - 1)/2$ for odd f. And **stride 2 halves it**, which is the cheap alternative to pooling.

3.3 Kernels chosen by hand, and the zero-sum trick

Before letting a network learn kernels, it is worth writing a few down. A kernel is nine numbers; the question is what arrangement of nine numbers responds to one thing and ignores another. Four are enough to make the point — and only one of them finds the defect we care about.



The same die under four kernels written down rather than learned: a local average, two edge detectors, and a centre-surround contrast detector. Only the last one isolates the scratch.

The last kernel is

$$K = \begin{pmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{pmatrix}$$

and its weights sum to zero. That is not decoration. If a patch is **constant** with value v, the response is

$$\sum_{i,j} v \cdot K[i,j] = v \sum_{i,j} K[i,j] = v \cdot 0 = 0$$

whatever v is. A zero-sum kernel is therefore **blind to absolute brightness** and responds only to local contrast — which is exactly what section 1.1 said this problem requires, since the film thickness varies across every die.

Measured in notebook 01: the strongest response on a defective die is **3.129**, against **1.118** on a clean one. Nine numbers, no training, and the hardest part of the problem is handled.

Notebook 02 comes back to this. Of the eight 3×3 kernels the first layer *learned*, seven sum to less than 0.6 in absolute value, against a scale of 2.16 if their weights all pointed the same way. Nothing required that. Gradient descent rediscovered the zero-sum trick because the data rewarded it.

---

4. Equivariance, and what pooling turns it into

4.1 The property the whole idea rests on

Convolution is equivariant to translation: shift the input and the output shifts by the same amount. The response does not change, it moves.

$$(\text{shift}_d\, I) * K = \text{shift}_d(I * K)$$

This follows from the definition in one line. Write I' for the shifted image, so that $I'[r,c] = I[r-d,c]$ — pixel r of the shifted image is pixel $r-d$ of the original. Then at every position

$$(I' * K)[r,c] = \sum_i I'[r+i,c]\, K[i] = \sum_i I[r-d+i,c]\, K[i] = (I * K)[r-d,c]$$

which is the output shifted by d, and nothing else has changed: **the same f pixels meet the same f weights, only at a different address.** That is the whole property, and it holds because the kernel is the same at every position — which is exactly what a dense layer does not do.

It is still worth checking rather than believing. Notebook 01 computes both sides at four different offsets and, away from the borders where the shift wraps around, they agree **exactly, to the last bit**: the largest difference is 0, not merely small.

Top: the same die shifted by four different amounts. Bottom: each one's response to the contrast kernel. The bright spot tracks the defect, and no retraining occurs between panels because there is nothing to retrain.

A dense layer has no such property, and this is the difference the whole lesson turns on. Shift the input by one pixel and every one of a unit's 576 weights now multiplies a different pixel. The unit's output is not shifted; it is unrelated.

4.2 Pooling: from *where* to *whether*

Equivariance says the response moves with the defect. For **grading a die** we do not want the response to move — we want one number saying whether a strong response occurred *anywhere*. Max pooling takes the maximum over a window and discards the position:

$$P[r, c] = \max_{0 \le i, j < k} A[rk + i,\ ck + j]$$

A feature map pooled three times, 24×24 down to 3×3. The maximum is 3.129 at every stage — which is the point of taking a maximum.

Pooling does three things, and they are worth separating:

1. **It converts equivariance into invariance.** Move the defect a pixel and after pooling the output is often literally identical.
2. **It enlarges the receptive field.** After a 2×2 pooling each cell summarises two pixels in each direction, so a 3×3 kernel in the next layer spans 6×6 of the map before pooling — and, with this lesson's first 3×3 layer beneath it, $6 + 2 = 8$, an 8×8 window of the original image, for the same nine weights. Stack enough and a small kernel sees most of the die.
3. **It discards spatial precision**, which is a cost, not a benefit, whenever *where* is part of the answer — segmentation, detection, keypoints. For a pass/fail grade it is free.

Global max pooling, used in this lesson's architecture, takes the maximum over the whole map: one number per kernel, saying whether that kernel ever fired strongly anywhere on the die. That single design choice is what makes the network's answer independent of position, and section 7 is what it buys.

---

5. A convolutional network, and what it costs

The architecture, in full: two convolutional layers with pooling, a global maximum, and a small dense head.

layer	output shape	parameters
input	24 × 24 × 1	0
conv, 8 kernels of 3×3, same padding	24 × 24 × 8	80
max pool 2×2	12 × 12 × 8	0
conv, 16 kernels of 3×3, same padding	12 × 12 × 16	1,168
max pool 2×2	6 × 6 × 16	0
global max pool	16	0
dense, 16 units	16	272
dense, 1 unit, sigmoid	1	17
	total	**1,537**

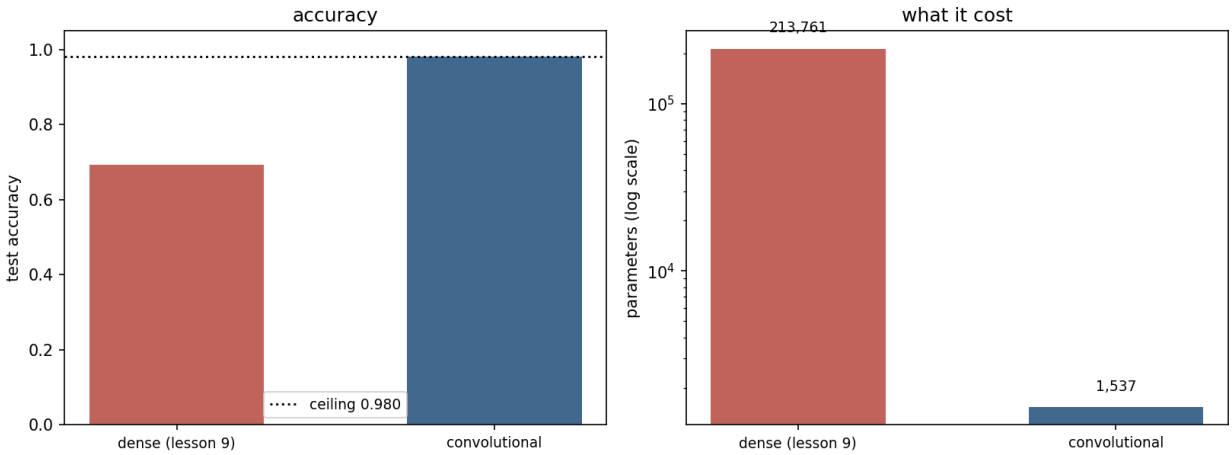
Check the second convolutional layer's count, because it is the one students get wrong: each of its 16 kernels is $3 \times 3 \times 8$ — three by three *across all eight input channels* — so $16 \times (9 \times 8) + 16 = 1{,}168$. A kernel's depth always matches the input's channel count, and only its two spatial dimensions are chosen. A third layer of 32 such kernels on these 16 channels would cost $32 \times (9 \times 16) + 32 = 4{,}640$. The dense head at the end is only 272 + 17 = 289 parameters, because it sits after the global pooling rather than on the pixels.

Against it, a dense network of the kind Lesson 9 built, sized for these images: two hidden layers of 256 units on the 576 input pixels, then one output — $576 \times 256 + 256$, then $256 \times 256 + 256$, then $256 + 1$, which is **213,761** parameters. (Lesson 9's own digit networks were smaller because its images were: one hidden layer of 32 or 64 units on 8×8 inputs, 2,410 and 4,810 parameters. The architecture is the same idea at a different scale.)

5.1 Head to head

Trained on 3,000 images, tested on 2,000, ceiling 0.9800:

network	accuracy	sd over seeds	parameters
dense (lesson 9)	0.6928	0.0342	213,761
convolutional	**0.9800**	0.0000	1,537



The same images, the same optimiser, the same epoch budget. The right panel is on a logarithmic scale.

Now the number that matters more than the accuracy:

scored against	convolutional network
the recorded grade	0.9800
the true grade	**1.0000**
recorded vs true	0.9800

The convolutional network is right about every single die. Its 0.98 is not a modelling shortfall at all — it is the grading station's error rate, in full, and nothing else. There is no gap left for a better architecture to close, and any effort spent trying to reach 0.99 would be effort spent learning to reproduce the station's mistakes.

This is the identity behind lesson 9's ceiling of 0.97, and it holds for the same reason. If a model agrees with the truth on a fraction q and the station records the wrong grade independently with probability e:

$$\text{accuracy against the recorded grade} = q(1-e) + (1-q)e$$

At $q = 1$ and $e = 0.02$ this gives exactly 0.98.

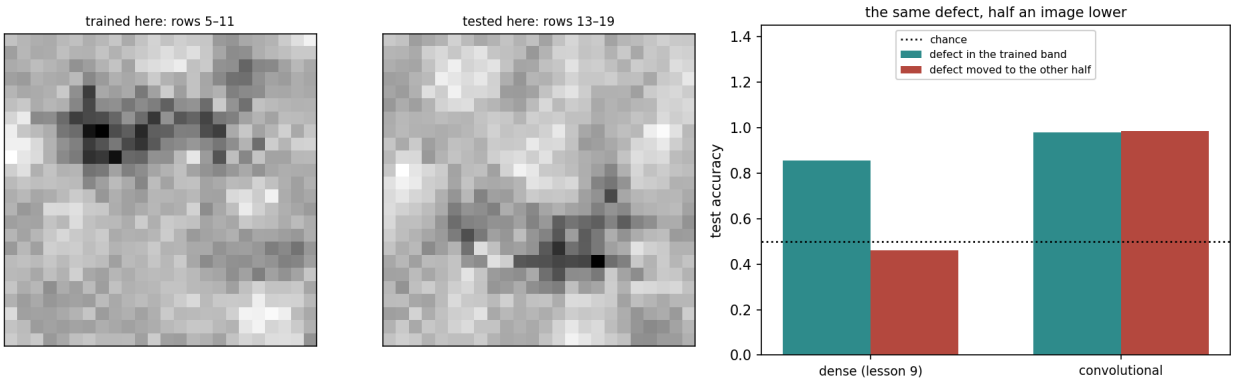
---

6. The experiment this lesson exists for

Everything so far is an argument. This is the measurement.

Defects are confined to a **band of rows**: the top of the die for training, the bottom for one of the two test sets. Nothing else changes — same generator, same defect, same contrast, same number of images. The model is then asked about a position where it has never seen a defect.

network	defect in the trained band	defect moved to the other half	cost
dense (lesson 9)	0.8567	**0.4617**	**−0.3950**
convolutional	0.9800	0.9847	+0.0047



Left and centre: forty defective dies from each band, averaged, so the bright smear shows where the defects were. Right: the result.

The dense network falls below chance. It scored 0.86 while the defect stayed where it had been seen, and moving that identical defect half an image down destroys it — not degrades it, destroys it. Every weight carrying the evidence is attached to a specific pixel, and those pixels are now different pixels.

The convolutional network does not move at all. It was never told where to look, because the question it asks is asked everywhere.

This is what an **inductive bias** is, made measurable. And the direction of the argument is the opposite of the intuitive one, which is why it is worth stating carefully:

> A convolutional layer is a **restricted** dense layer. Every function it can compute, a dense layer can compute too — take the dense weight matrix, tie the entries that a shared kernel would tie, and set the rest to zero. It is strictly *less* expressive.

The restriction is the whole value. The functions it gives up are precisely the ones that treat row 5 differently from row 15, and on this problem nobody wanted any of those. **A model that cannot express a wrong answer does not have to learn to avoid it.**

6.1 The predictable mistake

Most students, asked in advance, predict that the dense network will do *worse* on the moved band but stay well above chance — that it has learned something about scratches in general and will retain part of it.

The reasoning is sound, and it is how almost every other kind of distribution shift behaves: performance degrades, it does not vanish. What makes this case different is that the shift is not a

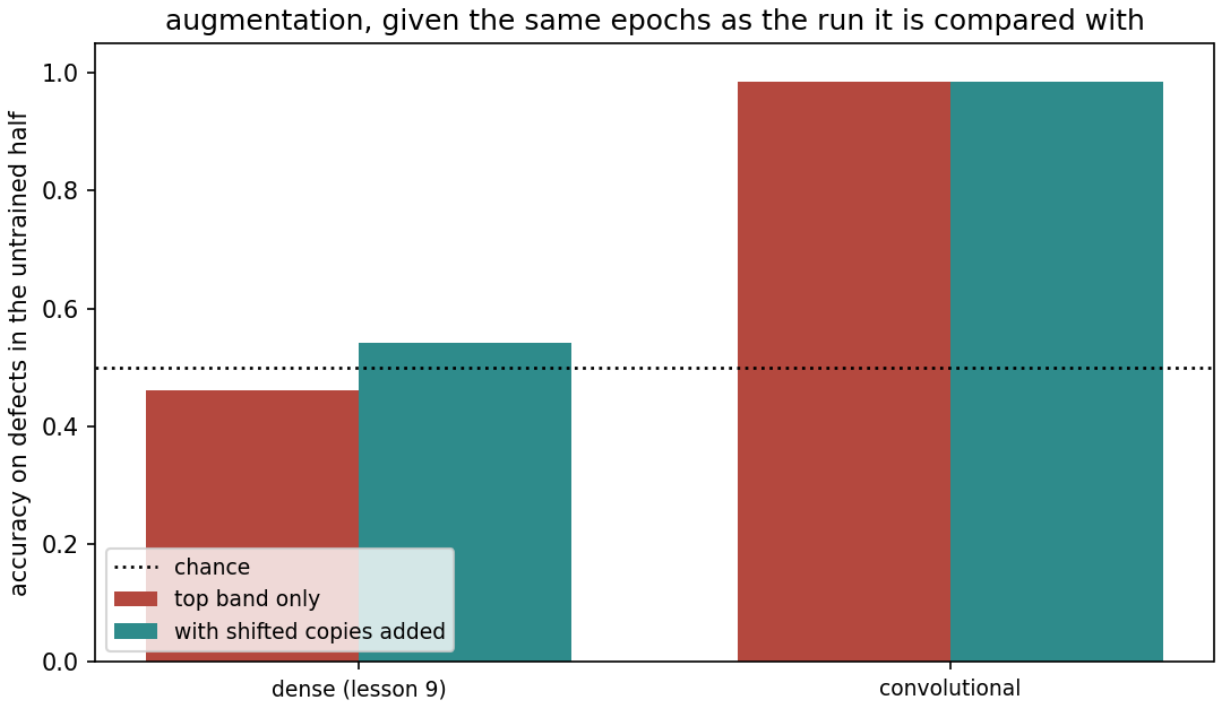
change of *degree* in the inputs but a change of *which inputs carry the signal*. The dense network's knowledge is stored per position, and the positions in question were never trained. There is nothing to degrade gracefully.

---

7. Augmentation against architecture

If the dense network's problem is that it never saw a defect low on the die, one obvious repair is to show it some, by shifting the images it already has. **Data augmentation** applies transformations the label is known to survive: a scratch moved three pixels is still a scratch.

network	trained on the top band only	with shifted copies added	gain
dense (lesson 9)	0.4617	0.5423	**+0.0807**
convolutional	0.9847	0.9847	0.0000



Five times the training data, every copy shifted by its own random offset, and the same epoch budget as the run it is compared against.

Eight points: real, measurable, and still **forty-four points short** of what the architecture delivers for free.

Two details of that comparison are worth copying rather than the result itself, because both are easy to get wrong in a way that flatters the conclusion you were hoping for.

Both arms got the same number of epochs. An augmented set is five times larger, so giving both the same wall-clock budget would have quietly handed the augmented one five times fewer

passes over each image. An earlier version of this experiment did exactly that and reported a gain of 1.6 points; equalising the epochs turned it into 8.1.

The shifts are drawn per image, not per batch. One offset applied to a whole copy adds four more fixed positions — four more special cases, not augmentation.

The distinction to carry away:

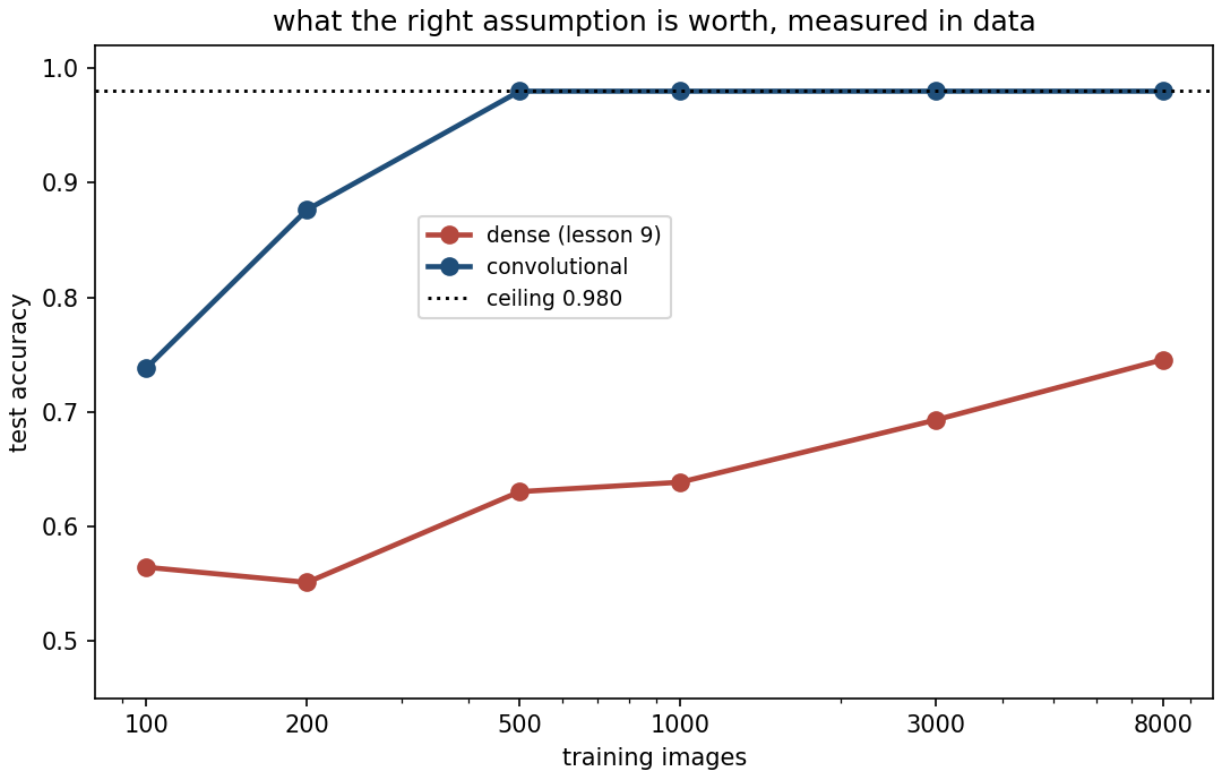
	augmentation	architecture
how the invariance arrives	taught, one example at a time	asserted
cost	training data and capacity	none
exactness	approximate	exact
can it be forgotten?	yes	no

Use augmentation for invariances **no layer can express** — brightness, contrast, small rotations, elastic distortion — where it is indispensable. It is a poor substitute for one that a layer *can* express, and translation is the canonical example of the latter.

---

8. What the assumption is worth, measured in data

training images	dense (lesson 9)	convolutional
100	0.5643	0.7383
200	0.5510	0.8765
500	0.6303	**0.9800**
1,000	0.6385	0.9800
3,000	0.6928	0.9800
8,000	0.7455	0.9800



Test accuracy against training-set size, logarithmic horizontal axis.

The convolutional network reaches the ceiling at 500 images and stays there. The dense network, at 8,000 — sixteen times more data — is still 0.235 short, and the gap it is closing is the one weight sharing removed at the start.

That flat line is not a plotting error, and it is worth saying so in class because it looks like one. The convolutional network is already right about every die; the only thing left between it and 1.0 is the station.

---

9. What is the convolution actually using?

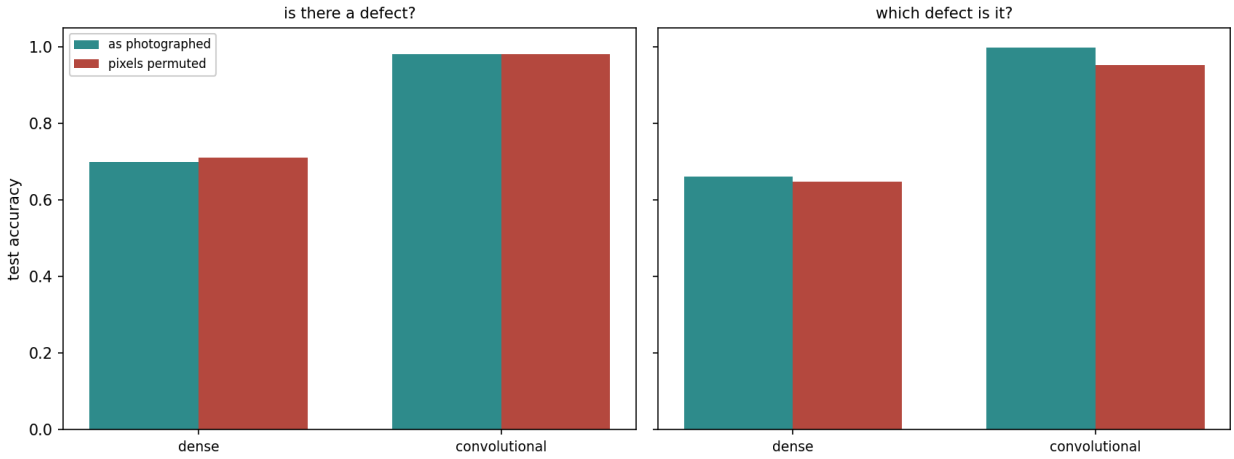
Convolution is usually justified in one breath: *images have spatial structure, convolutions exploit spatial structure*. That sentence bundles two separate assumptions, and this section takes them apart with a measurement.

Apply **one fixed permutation** to the pixel positions of every image, then train and test on the shuffled version. Lesson 9's closing paragraph says the dense network cannot notice. What should happen to a convolution, whose kernels are defined entirely by adjacency?

Run it on two tasks: grading a die **defective or not**, and naming **which of three types** of defect it carries.

task	network	as photographed	pixels permuted
is there a defect?	dense	0.6985	0.7097

task	network	as photographed	pixels permuted
is there a defect?	convolutional	0.9798	**0.9800**
which defect is it?	dense	0.6615	0.6478
which defect is it?	convolutional	0.9970	**0.9523**



Read the pairs of bars, not the heights. Everywhere except the last pair the permuted bar matches the photographed one, which means the arrangement of the pixels was never being used. Only the convolutional network on "which defect is it?" loses anything at all, and that single 4.5-point drop is the entire measured value of locality on this problem.

Three readings, and the middle one is the surprise.

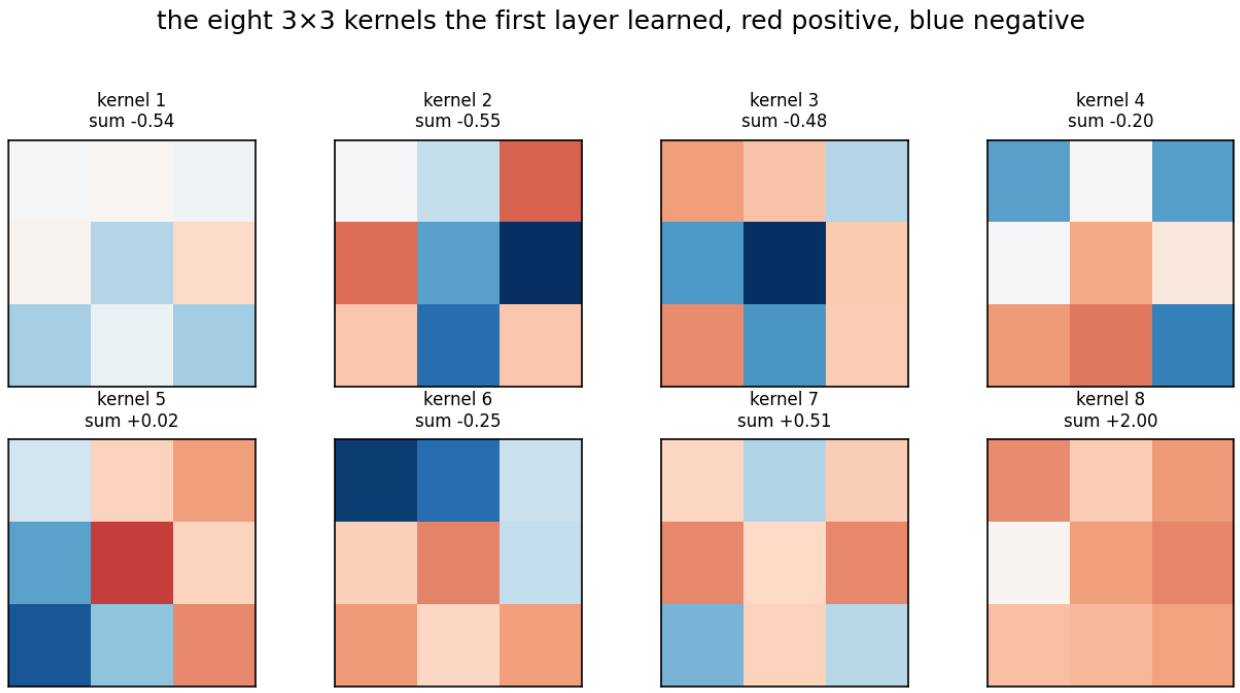
The dense network does not notice, on either task. Exactly as lesson 9 predicted: for a dense layer a permutation is a relabelling of input units and nothing more.

On "is there a defect?", the convolution does not notice either. That looks wrong until you ask what the task actually requires. To grade a die you must answer "is there a patch of unusual local contrast *somewhere*", and a scattered permutation still leaves those pixels somewhere, still extreme. The convolution is winning this task on **weight sharing alone**. It never needed the arrangement.

On "which defect is it?", it notices — 4.5 points. Telling a scratch from a particle is a question about *shape*, a line against a blob, and shape is what the permutation destroys.

assumption	what it says	which task needed it
weight sharing	a pattern means the same thing wherever it appears	both
locality	nearby pixels belong together	only the typing task

The usual one-line justification is true but blurred. On this data most of the advantage came from the first assumption, and only naming the type required the second. When you reach for a convolution, it is worth knowing which of the two you are actually buying — because if it is only weight sharing, other architectures supply that too.

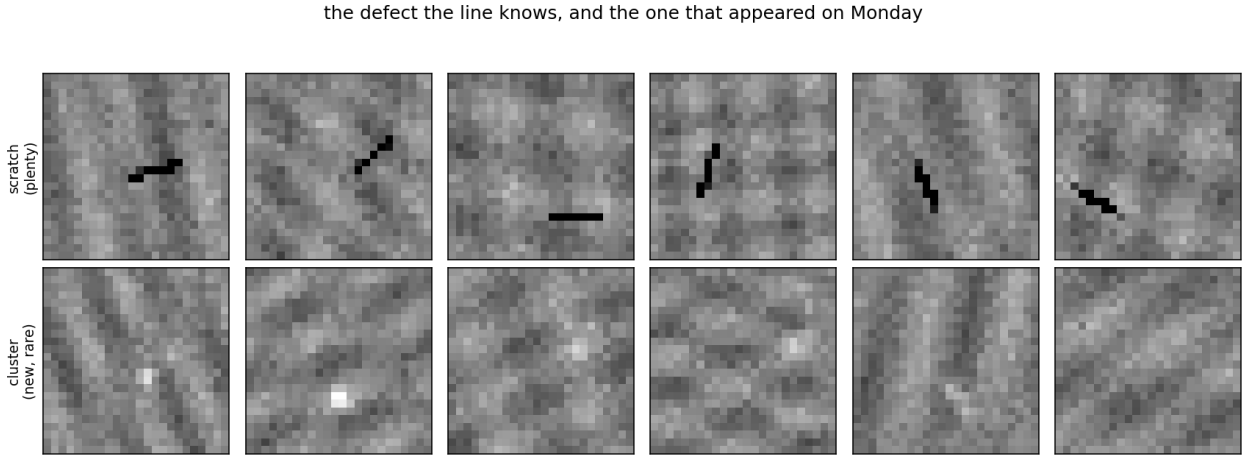


The eight 3×3 kernels the first layer learned, red positive, blue negative. Seven of the eight sum to nearly zero: gradient descent rediscovered section 3.3's zero-sum trick without being told.

---

10. Transfer learning

A new defect appears on Meridian's line on a Monday — a contamination **cluster**, several faint specks together. By Friday there are a few dozen labelled photographs. Section 8 says the convolutional network needs several hundred, and waiting six months is not an answer because the line is shipping now.



The defect the line knows, and the one that appeared on Monday.

Transfer learning starts the new network from the weights of one trained on a related task, on the argument that early layers detect local contrast and nothing about that is specific to scratches.

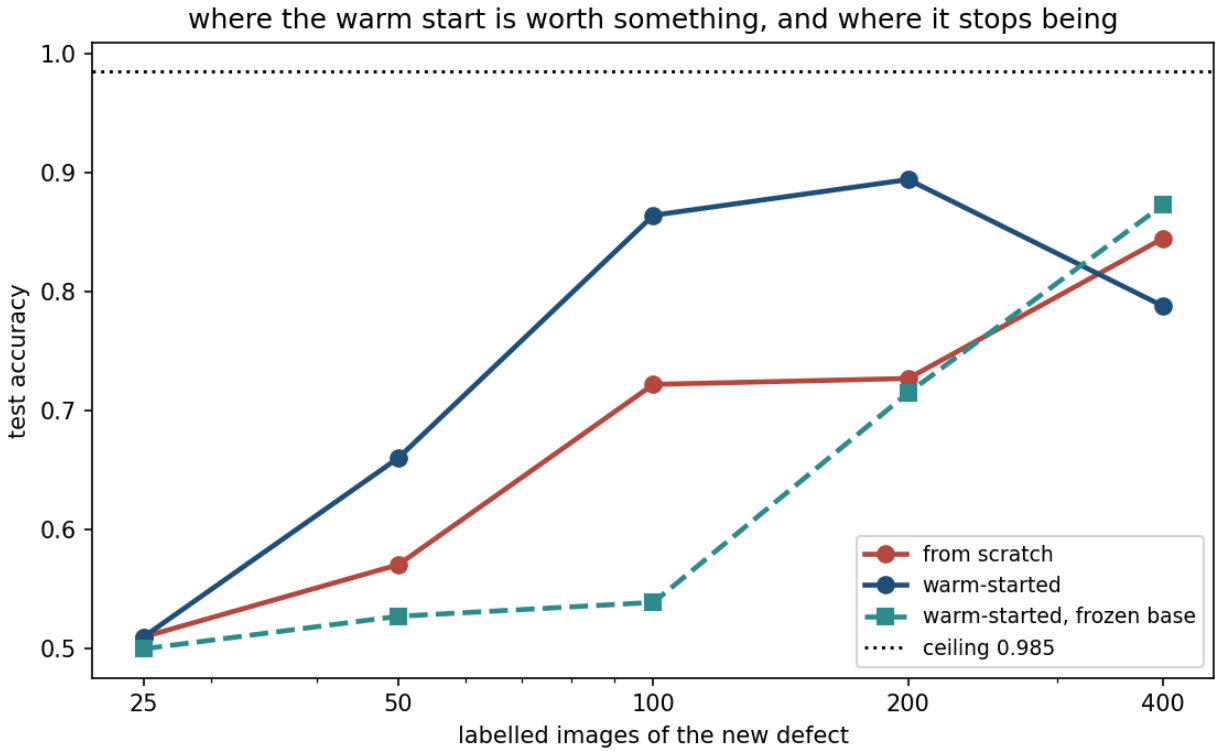
10.1 The source task is the whole game

This lesson pre-trains on a task **nobody at Meridian wants solved**: naming which of three types — clean, scratch, particle — a die carries. Grading pass/fail on scratches alone teaches *one* detector. Naming the type forces the early layers to describe local structure in general, dark lines and bright blobs both, and those are the features worth carrying.

The pre-training task reaches 0.9993. That number is not a result; it is a receipt showing the source task was learned, which is the precondition for its features being worth anything.

10.2 A window, not a slope

labelled images	from scratch	warm-started	frozen base	gain from warm start
25	0.5100	0.5100	0.5000	0.0000
50	0.5707	0.6603	0.5273	+0.0897
100	0.7220	**0.8640**	0.5390	**+0.1420**
200	0.7270	**0.8940**	0.7150	**+0.1670**
400	0.8447	0.7877	0.8727	−0.0570



Look at the gap between the blue line and the red one, not at either line alone. It is zero at 25 images, widest at 200, and has closed and reversed by 400 — the shape of a window, not of a slope. The dashed line is the frozen base, which never competes until the very end.

The received wisdom is that transfer helps most when data is scarcest. **These numbers do not say that**, and the shape they do describe is more useful:

- **At 25 images nothing works.** Both arms sit at chance. There is too little to fit even a head onto good features.
- **Between 50 and 200 the warm start is worth 9 to 17 points**, and the gain *grows* across that range rather than shrinking.
- **By 400 it has stopped helping.** The from-scratch network has now seen enough of the new defect to learn its own features, and the borrowed ones are a constraint rather than a head start.

Transfer occupies a **window**: too little data and it has nothing to attach to, too much and it has nothing to add.

10.3 When transfer makes things worse

Repeat the experiment with a source chosen badly — pre-train only on scratches, which are dark lines, and transfer to clusters, which are bright specks. With the base frozen so the borrowed features must stand alone:

labelled images	from scratch	frozen, from the typing task	frozen, from scratches only
50	0.5707	0.5273	0.5150
100	0.7220	0.5390	0.5047
200	0.7270	0.7150	0.5350

Two findings, and the second was not planned.

The badly chosen source never leaves chance. Its kernels are committed to reporting darkness where the new defect is bright, and a frozen layer cannot change its mind. This is **negative transfer**, and it is real.

But freezing hurt the good source too. The typing features, frozen, are also behind starting from noise at 50 and 100 images, and only pull level by 200 — while the *unfrozen* warm start from the same source was 14 points ahead at exactly 100. The features were useful; freezing them was the mistake.

So the rule is neither "always pre-train" nor "freeze the base":

- **Transfer carries whatever the source task forced the network to learn, and nothing else.** Ask what the source actually required before assuming its features generalise.
- **Freezing is a bet that the borrowed features are already right.** It is cheap, and safe only when source and target are close. When in doubt, warm start and leave everything trainable: that recovers from a bad source, and a frozen base cannot.

---

11. A note on what this lesson did not need

Everything above ran on a laptop-class processor with no graphics card, on images of 576 pixels, with a network of 1,537 parameters. That is deliberate, and it is worth naming what was skipped rather than pretending the field ends here.

Real convolutional networks are deeper, and depth reintroduces exactly lesson 9's problem: a stack of layers attenuates gradients. The repairs are **batch normalisation**, which renormalises each layer's activations so the signal neither dies nor explodes, and **residual connections**, which add a layer's input to its output so that the gradient has a path that skips the layer entirely. Both are answers to lesson 9's vanishing gradient (its section 6.4), applied at depth.

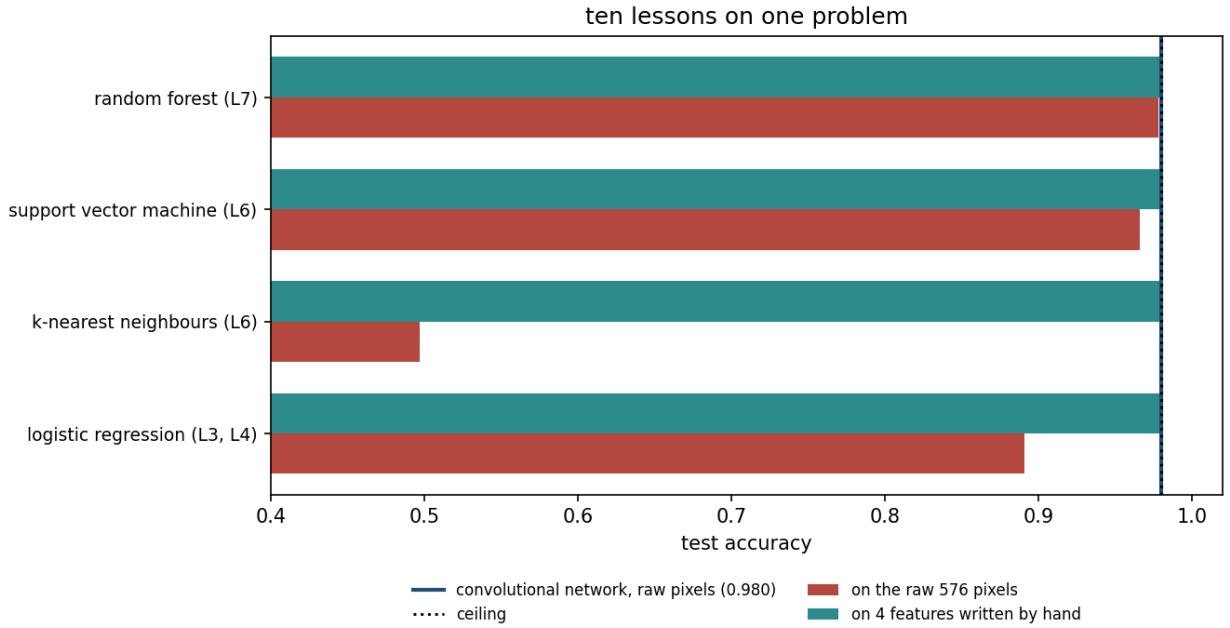
They are not covered here because 24×24 images with one defect do not need them, and a technique introduced without a problem it solves is a technique nobody remembers. `Resources/` carries a pointer.

---

12. Ten lessons on one problem

Every family the course has met, on the wafer images. First on the raw 576 pixels; then on **four numbers per image**, computed with the contrast kernel written down by hand in notebook 01 before anything was trained.

family	on raw pixels	on 4 hand-made features
logistic regression (lessons 3, 4)	0.8905	0.9800
k-nearest neighbours (lesson 6)	**0.4970**	0.9800
support vector machine (lesson 6)	0.9660	0.9800
random forest (lesson 7)	**0.9780**	0.9795
dense network (lesson 9)	0.7430	—
convolutional network (lesson 10)	**0.9800**	—



Ceiling 0.9800. The blue line is the convolutional network on raw pixels.

This table was written expecting to show that classical methods cannot handle images. **It shows no such thing**, and the honest reading is more interesting than the one it was meant to produce.

A random forest reaches 0.9780 on raw pixels — level with the convolutional network, knowing nothing about which pixel is next to which. It does not need to. A defect drives *some* pixel to an unusual value, and 300 trees splitting on individual pixels cover enough of them to notice. That is the forest's own inductive bias — "the answer depends on a few features crossing thresholds" — and it happens to suit this task.

Two families fail, instructively. k-nearest neighbours collapses to 0.4970 because Euclidean distance over 576 pixels is dominated by the smooth background, and a seven-pixel defect barely moves it: lesson 6's curse of dimensionality, exactly. And **lesson 9's dense network, at 0.7430, sits behind plain logistic regression.** Flexibility without a matching assumption is not an advantage; it is a larger space to search for the same answer.

Four hand-written numbers put every classical family at the ceiling. The convolutional network, given only raw pixels, found an equivalent feature by itself.

That is the summary of the ten weeks, and it is about representations rather than models:

> A model turns a representation into a decision. **The representation usually matters more than the model.** Classical methods make you build it; convolutional networks learn it, paying in data and compute for the privilege. Neither is a default.

Which is why lesson 9 found a hidden layer worth 39 points on one problem and 0.09 on another; why this lesson's convolution is worth 24 points over a dense network here; and why a random forest given nothing but raw pixels came within two thousandths of it.

12.1 Choosing, in one table

If	then
the inputs have a known geometry — image, audio, time	convolution, and say which of weight sharing or locality you need
a pattern means the same thing wherever it appears	weight sharing earns its cost
the answer depends on a few features crossing thresholds	a tree ensemble, and it may be unbeatable
you can write the right feature down yourself	do that first, and check whether anything beats it
distances between whole examples are meaningful	k-nearest neighbours; check the dimensionality first
labels are scarce and a related task is not	transfer, warm start, do not freeze
you cannot say what assumption your model encodes	you have not chosen a model, you have chosen a default

12.2 And after this

The programme continues with **Toolkit for Modern Algorithms**, on large language models and agents, which this course deliberately left alone.

Very little of what you did over these ten weeks stops being useful there, and some of it is not even renamed: the loss a language model trains on is lesson 4's, its last layer is lesson 9's softmax,

fetching documents before answering is lesson 6's k-nearest neighbours, and sampling an answer several times and voting is lesson 7's bagging with lesson 7's $\rho\sigma^2$ floor.

`Course/BRIDGE.md` sets that out in full, along with the part that matters more: the methodology. Nobody teaches it twice, and it is what separates being able to build one of these systems from being able to say whether it is any good.

Summary

- A convolutional layer applies **the same** kernel everywhere. The saving in parameters is a side effect; the point is that one example teaches the detector at every position.
- 1,537 parameters against 213,761 — a factor of 139 — and the small one scores **1.0000 against the true grade** — its visible 0.9800 is the grading station's error and nothing else.
- **Move a defect to a half of the die where none was seen in training and the dense network falls below chance, 0.8567 to 0.4617, while the convolutional one does not move.** That is the number to remember: an inductive bias is worth more than expressiveness, because convolution can express strictly *fewer* functions and the ones it gives up were all wrong.
- Augmentation bought the dense network 8 points and left it 44 short. It teaches an invariance; architecture asserts one.
- The convolutional network reaches the ceiling at 500 images; the dense one is 0.235 short at 8,000.
- Permuting the pixels costs the convolution **nothing** on detection and 4.5 points on typing: weight sharing and locality are two assumptions, and only the second concerns arrangement.
- Transfer occupies a **window** — nothing at 25 images, +17 points at 200, negative by 400 — and freezing a base is a bet that the borrowed features are already right.
- On raw pixels a random forest matched the convolutional network and a dense network lost to logistic regression. **The representation is doing most of the work; the only question is who builds it.**

Homework

Exercise 10 — see `Exercises/10_convolutional_networks.md`. It is the one exercise with no lesson after it to discuss it, so it is worked for the exam alone. It is the last exercise of the course, and the one you will have had least practice defending; write the reasoning down as you go.

Notation used in this lesson

Symbol	Meaning
I, K	the input image and a kernel
$I * K$	the convolution (strictly, cross-correlation) of the two
n, f	input size and kernel size, per spatial dimension
p, s	padding and stride
q, e	a model's agreement with the truth; the station's error rate
$\hat{y}$, y	prediction, target

Symbol	Meaning
α	learning rate, and nothing else

Further reading

Source	Why
LeCun et al., "Gradient-based learning applied to document recognition", *Proc. IEEE* 86 (1998)	The paper that made this architecture, and still the clearest statement of why
Goodfellow, Bengio & Courville, *Deep Learning* (2016), ch. 9	Convolution as an infinitely strong prior — section 6's argument, in full
Zeiler & Fergus, "Visualizing and understanding convolutional networks" (2014)	What the layers actually learn, beyond the first
He et al., "Deep residual learning for image recognition" (2016)	Section 11's repair for depth
Yosinski et al., "How transferable are features in deep neural networks?" (2014)	Section 10 measured properly, layer by layer
Ioffe & Szegedy, "Batch normalization" (2015)	The other half of section 11